

Genossi

Audit Log Integrity and Compliance

Technical documentation for auditors and operators

Contents

1	Purpose and Scope	1
1.1	Reference frameworks	1
2	Architecture Overview	2
3	Contents of the Audit Log	3
3.1	Protected entity types	3
3.2	Logged operations	3
3.3	Fields of an audit entry	3
4	Tamper-Evidence Through the Hash Chain	5
4.1	Principle	5
4.2	Hash computation	5
4.3	Internal verification	5
5	Qualified Electronic Time Stamp	6
5.1	Purpose	6
5.2	Workflow	6
5.3	Interval	6
5.4	Choice of TSA	7
6	Roles and Access	8
7	Retention and Export	9
7.1	Database	9
7.2	External mirroring (optional)	9
8	Verification by an Auditor	10
8.1	1. Internal verification	10
8.2	2. Inspection of time-stamp entries	10
8.3	3. External verification of the TSR token	10
9	Operator Configuration	11
10	Implementation Boundaries	12
11	Appendix A – Glossary	13
12	Appendix B – References	14

1 Purpose and Scope

This document describes the mechanisms by which the **Genossi** software ensures that write-operations on business-relevant data are traceable and tamper-evident. It is addressed to:

- **Auditors** (in particular cooperative auditors under the German § 53 GenG, but also compliance, data-protection and IT-security auditors) wishing to understand the control framework;
- **Operators** of the software wishing to provide evidence to third parties (audit association, supervisory board, external audit) of the controls in place.

The presentation is deliberately restricted to aspects that are relevant to the audit. Detailed implementation guidance and developer notes can be found in the project's source documentation.

1.1 Reference frameworks

The mechanisms described below align with the following frameworks:

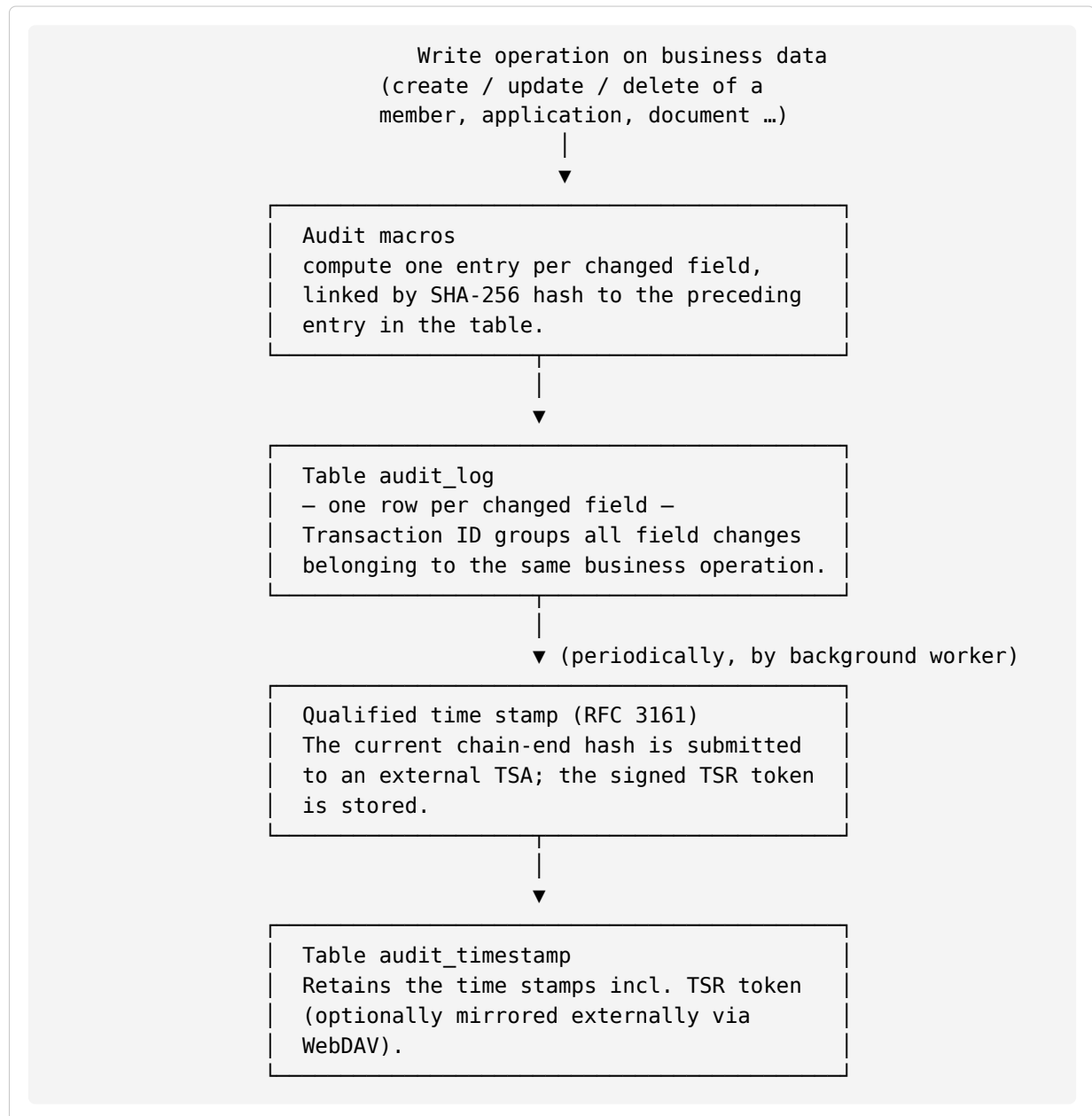
GoBD	German principles for the orderly keeping and retention of books, records and documents in electronic form, as well as for data access (issued by the Federal Ministry of Finance)
§ 239 HGB	Requirements under the German Commercial Code concerning the keeping of commercial books (including immutability and traceability)
eIDAS Regulation	Regulation (EU) No 910/2014, in particular Articles 41 and 42 (qualified electronic time stamps)
RFC 3161	Internet X.509 Public Key Infrastructure Time-Stamp Protocol — the technical protocol used for exchanges with time-stamping authorities
§ 53 GenG	Mandatory audit of registered cooperatives under the German Cooperatives Act

Summary

Changes to protected entities are recorded in a cryptographically linked hash chain (SHA-256). At regular intervals the current end of the chain is bound to an external trust service by a **qualified electronic time stamp** under eIDAS. This combination makes subsequent tampering or back-dating of records provably detectable.

2 Architecture Overview

The audit function is structured in several layers and uses established standards exclusively (SHA-256, ISO 8601, RFC 3161):



The separation between **internal integrity** (the hash chain) and **external anchoring** (the qualified time stamp) is deliberate: the chain detects any subsequent modification within the database, whereas the time stamp prevents a tampered chain from being silently recomputed to a seemingly consistent earlier state.

3 Contents of the Audit Log

3.1 Protected entity types

Write operations on business data are logged. The following entity types are currently subject to audit logging:

- **Member** — member records of the cooperative
- **MemberAction** — actions performed on a member
- **MemberDocument** — documents associated with a member
- **Application** — membership applications

Additional entities can be added by implementing the internal `Auditable` trait and using the corresponding audit macros. The mechanism is designed for future extension.

3.2 Logged operations

<code>create</code>	Creation of a new record
<code>update</code>	Modification of existing field values
<code>delete</code>	Deletion (implemented as a soft delete via a deletion timestamp)
<code>snapshot</code>	Full representation of a record (administrative retro-documentation)

3.3 Fields of an audit entry

Each row in the `audit_log` table corresponds to exactly one modified field and contains:

<code>id</code>	Unique identifier of the audit entry (UUID)
<code>timestamp</code>	Time of recording in UTC, ISO 8601
<code>user_id</code>	Login identity of the triggering user (for system actions: <code>SYSTEM</code>)
<code>process</code>	Name of the triggering process (e.g. <code>member-service</code> , <code>audit-snapshot</code>)
<code>transaction_id</code>	UUID grouping all field changes of a single business operation
<code>entity_type</code>	Type of the affected entity (e.g. <code>member</code>)
<code>entity_id</code>	UUID of the affected entity
<code>action</code>	<code>create</code> / <code>update</code> / <code>delete</code> / <code>snapshot</code>
<code>field_name</code>	Name of the changed field
<code>old_value</code>	Previous value (empty for <code>create</code>)
<code>new_value</code>	New value (empty for <code>delete</code>)
<code>prev_hash</code>	Hash value of the preceding audit entry
<code>entry_hash</code>	Hash value of this entry (see section 4)

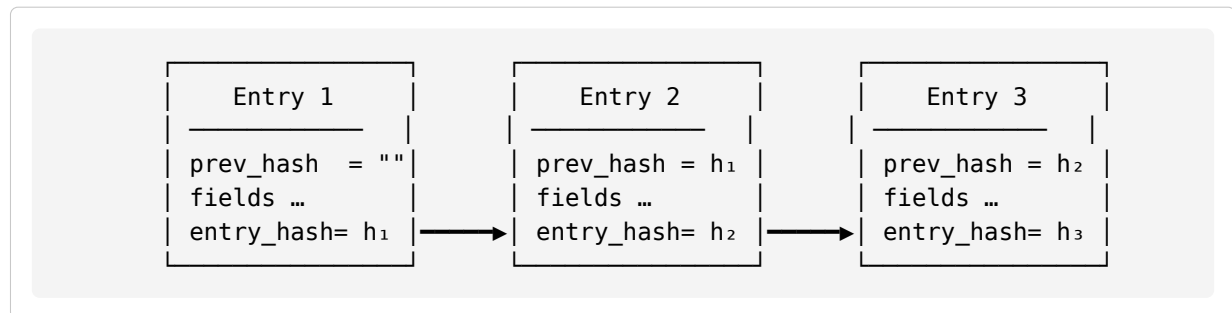
Granularity

One **separate** row is written for each modified field. An operation that changes three fields of a member record produces three rows sharing the same `transaction_id`. This enables field-level reconstruction of the change history.

4 Tamper-Evidence Through the Hash Chain

4.1 Principle

Audit entries form a linear chain: the `entry_hash` of one row becomes the `prev_hash` of the **next** row. Any subsequent modification to a previously written entry destroys the consistency of all following hashes and is therefore unambiguously detectable.



Tampering with e.g. the `new_value` of Entry 2 would change `h2`. Since `h2` is fixed as `prev_hash` in Entry 3, Entry 3 would also have to be recomputed, and so on for every subsequent entry. Without simultaneously rewriting **all** later entries **and** all affected time-stamp records (see section 5), any tampering is immediately detectable.

4.2 Hash computation

The `entry_hash` is computed as the SHA-256 digest over a canonical representation of all entry fields. Inputs, in fixed order:

1. Timestamp (ISO 8601, UTC)
2. User identity (`user_id`)
3. Process name
4. Transaction UUID
5. Entity type
6. Entity UUID
7. Operation (`action`)
8. Field name
9. Previous value
10. New value
11. Hash of the preceding entry (`prev_hash`)

SHA-256 corresponds to the current state of the art for cryptographic hash functions and is considered secure by the German BSI.

4.3 Internal verification

The software provides a `verify_chain` function that, for a set of audit entries, checks

- that each entry carries a stored `entry_hash` which can be recomputed from the listed fields, and
- that the `prev_hash` values form an unbroken chain.

The REST endpoint `GET /api/audit/verify` performs this check over the entire chain and reports any “broken links” (incorrectly chained or tampered entries).

5 Qualified Electronic Time Stamp

5.1 Purpose

The hash chain alone proves that no **internal** inconsistency exists. An adversary with full database access could theoretically recompute the entire chain along with all hashes. To prevent this in practice, the current chain-end hash is periodically bound to an **external trust service**.

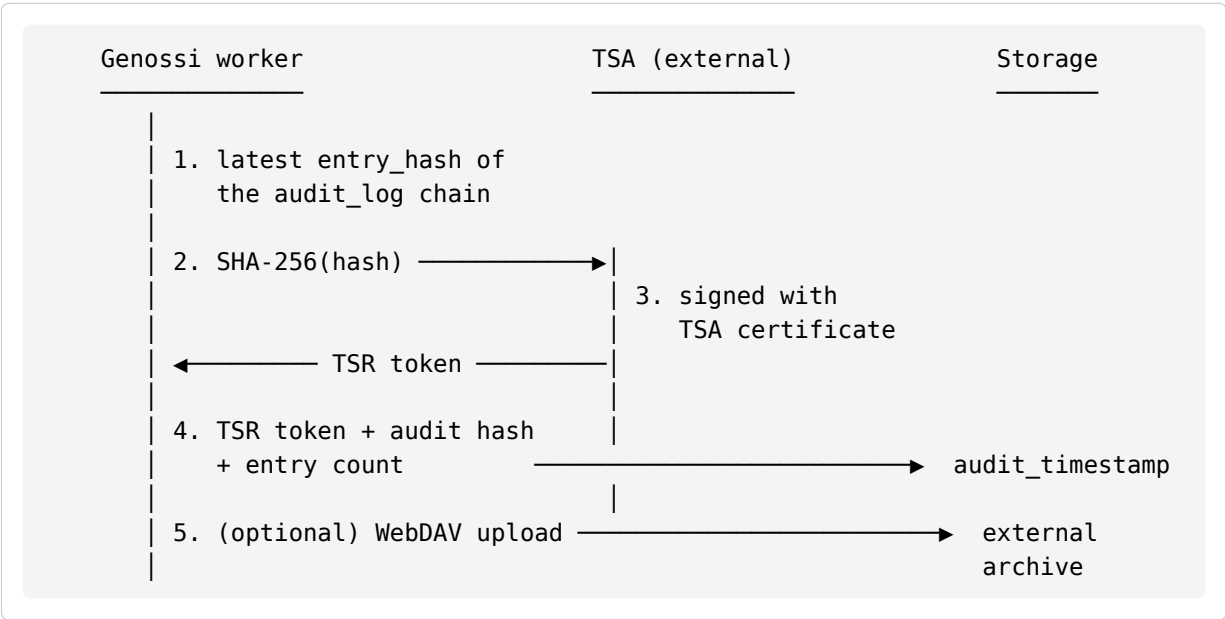
The software uses the **Time-Stamp Protocol** defined in RFC 3161. The external service (Time Stamping Authority, TSA) signs the submitted hash together with its own time signal. The resulting **TSR token** is legally secured by the TSA's certificate.

Where a TSA from the **EU Trust List** (LOTL) is used that is listed as a qualified trust service provider under eIDAS, the resulting time stamps are **qualified electronic time stamps** within the meaning of Article 42 of the eIDAS Regulation. They therefore enjoy the Union-law presumption of accuracy of time and integrity of the signed data.

Role of the TSA

The TSA never sees confidential data. Only a hash value is transmitted. No conclusions about the underlying audit entries can be drawn from this hash. Using an external TSA therefore does not breach any confidentiality obligations.

5.2 Workflow



Importantly, **plain-text business data never leaves the system**. The TSA receives nothing but a hash value.

5.3 Interval

The worker runs at an interval configured by the operator. The default is one week (168 hours). If the chain-end hash has not changed since the previous time stamp (for example because no write operations have occurred), no new TSA call is made and no costs are incurred. See section 9 for configuration details.

5.4 Choice of TSA

The choice of TSA is made by the operator. In order for the resulting time stamps to qualify as **qualified electronic time stamps** under eIDAS, the TSA must be listed in the EU Trust List. An up-to-date listing is provided at:

<https://eidas.ec.europa.eu/efda/tl-browser/>

The TSA actually used by a given installation is to be recorded in the **operator documentation** (see separate template `template-betreiber.de.pdf`).

6 Roles and Access

The audit system distinguishes between read and write privileges:

Regular users	Trigger audit entries through their business transactions but cannot view, modify or delete audit entries themselves.
Administrators	Can retrieve the complete audit log through the REST API and run integrity checks on chain and time stamps. They cannot modify or delete audit entries – the data model provides no such operation.
Technical system processes	The time-stamp worker runs as user <code>SYSTEM</code> and is only permitted to create new time-stamp records.

No deletion API

The software exposes neither REST endpoints nor service methods to modify or remove audit entries. Only a direct administrative intervention on the underlying database could alter an entry – any such intervention breaks the hash chain and is therefore detectable via the internal and external verification mechanisms.

7 Retention and Export

7.1 Database

Audit log entries and time stamps are stored in the same database as the business data. Data retention is therefore subject to the same rules as the remaining business data (operator's backup policy).

7.2 External mirroring (optional)

The software optionally transfers TSR tokens to an external store (WebDAV) after successful creation. This ensures that at least one complete time-stamp record resides outside the primary system, in keeping with the GoBD principle of holding “evidence outside the accounting system” for additional tampering protection.

The `audit_timestamp` table holds, for each time-stamp transaction, the basic metadata (time, associated hash, count of covered entries, status) together with the raw TSR token as a binary field. This token can be exported at any time for external verification.

8 Verification by an Auditor

An auditor has three mutually independent ways to verify the integrity of the audit chain. This independence is a deliberate feature of the control framework.

8.1 1. Internal verification

Chain verification (calling `GET /api/audit/verify`) can be triggered from the administration interface. The system reports whether the chain is consistent. This check can be performed at any time and incurs no cost.

8.2 2. Inspection of time-stamp entries

Each row in the `audit_timestamp` table shows:

- the point in time at which the time stamp was requested,
- the chain-end hash current at that point,
- the number of audit entries covered, and
- the status (`success` for a successful TSA response).

These fields can be displayed through the administration interface and printed as evidence.

8.3 3. External verification of the TSR token

For a cryptographic verification independent of the software, a TSR token can be exported and verified with an external tool. The **DSS Demo web application** of the European Commission is recommended:

<https://ec.europa.eu/digital-building-blocks/DSS/webapp-demo/>

The application validates the token against the EU Trust List (LOTL), verifies the TSA certificate signature and produces a formal validation report as PDF, which can be attached to the audit report.

External verification steps

1. Select a current time-stamp entry through the administration interface.
2. Export the associated TSR token as a file.
3. Upload the file to the DSS web application.
4. The application displays signature status, certificate chain and time and offers a signed validation report for download.

9 Operator Configuration

The software deliberately makes no assumption about the specific trust service provider in use. The operator configures:

<code>tsa_enabled</code>	Activation of the time-stamp function (<code>true</code> / <code>false</code>)
<code>tsa_url</code>	URL of the TSA endpoint (RFC 3161)
<code>tsa_user</code> (optional)	HTTP Basic Auth username
<code>tsa_pass</code> (optional)	HTTP Basic Auth password
<code>tsa_interval_hours</code>	Interval between time-stamp requests

The concrete values in a specific installation form part of the **operator documentation**, for which a template is provided.

10 Implementation Boundaries

A boundary of the current implementation is that the software's own `verify` function does not independently check the cryptographic **signature** of the TSR token against the TSA certificate; instead it verifies **chain consistency** and parseability of the token (TSA response status code).

The actual qualified cryptographic verification is deliberately delegated to an external, independent verification path (see section 8.3). This follows the information-security principle of **separating verification tools from production tools**: a potentially compromised system must not be both the producer **and** the sole verifier of its own evidence.

Note

For auditors: this boundary is transparently documented and architecturally intentional. Proof of the qualified signature — and therefore of the legal effect under Article 42 eIDAS — is provided through the external DSS web application of the European Commission.

11 Appendix A – Glossary

SHA-256	Standardised hash function (FIPS 180-4). Produces a 256-bit fingerprint of any input. Collision-resistant according to the current state of the art.
RFC 3161	Internet standard for time-stamp services. Defines the request format (“Time-StampReq”) and response format (“TimeStampResp” containing the TSR token).
TSA	Time Stamping Authority – trust service provider issuing time stamps according to RFC 3161.
TSR token	Time-Stamp Response token. Data packet signed by the TSA containing the submitted hash, a time stamp and the TSA signature.
eIDAS Regulation	EU Regulation No 910/2014 on electronic identification and trust services. Regulates, inter alia, qualified electronic time stamps (Articles 41–42).
EU Trust List (LOTL)	Official list of qualified trust service providers notified by EU member states.
GoBD	German principles for the orderly keeping and retention of books, records and documents in electronic form. BMF circular, most recent version 28 November 2019.
DSS	Digital Signature Service. Open-source European Commission tool for verifying electronic signatures and time stamps against the EU Trust List.
Hash chain	Data structure in which each entry contains a hash of the previous entry. Changes near the start of the chain invalidate all subsequent hashes and are therefore detectable.
Transaction UUID	Unique identifier grouping all audit entries belonging to one business operation, even though each field change is stored in a separate row.

12 Appendix B — References

- German Federal Ministry of Finance (BMF): **Principles for the orderly keeping and retention of books, records and documents in electronic form (GoBD)**, version of 28 November 2019.
- Regulation (EU) No 910/2014 of the European Parliament and of the Council of 23 July 2014 on electronic identification and trust services for electronic transactions in the internal market (**eIDAS Regulation**).
- IETF: **RFC 3161 — Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)**, August 2001.
- NIST: **FIPS PUB 180-4 — Secure Hash Standard**, August 2015.
- European Commission: **DSS Demo Web Application**, <https://ec.europa.eu/digital-building-blocks/DSS/webapp-demo/>.
- European Commission: **EU Trust List Browser**, <https://eid.ec.europa.eu/efda/tl-browser/>.
- Federal Law Gazette: **German Cooperatives Act (GenG)**, in particular § 53 (mandatory audit).